

Efficient Auto-Arbitrage Algorithms for Multi-Outcome Prediction Markets

June 28, 2026

Abstract

Multi-outcome prediction markets require auto-arbitrage to maintain the constraint that probabilities sum to one. Existing implementations use nested binary search, resulting in $O(n \log^2 N)$ complexity. We present three parameterization strategies—dollar-centric, share-centric, and probability-centric—that achieve $O(n \log N)$ complexity by eliminating the inner search layer. For typical precision $\epsilon = 10^{-6}$, this yields $\log N \approx 20\times$ speedup. Our approach is general: it works with any monotonic, computable cost curves (CPMM, log-scored AMMs, or hybrids with limit orders).

1 Introduction

Multi-outcome prediction markets with n mutually exclusive outcomes must maintain $\sum_{i=1}^n p_i = 1$. Trading in one outcome disrupts this sum, requiring *auto-arbitrage*: automatic rebalancing to restore the constraint.

Existing implementations use nested binary search: an outer search finds the auto-arbitrage amount, but each iteration requires inverting the cost function (shares $\leftrightarrow$ cost) via another binary search. This yields $O(n \log^2 N)$ complexity where $N \approx 1/\epsilon$ relates to precision.

Our contribution. Direct cost formulas eliminate the inner search, reducing complexity to $O(n \log N)$. We present three parameterization strategies for different use cases: spending a fixed budget, purchasing specific shares, or targeting a final probability.

2 Parameterization Strategies

2.1 The Design Space

Auto-arbitrage executes a purchase with rebalancing, computing the triple (s, c, p) of shares, cost, and probability. Each parameterization takes one element as input and solves for the others:

Strategy	Function Signature	Use Case
Dollar-centric	$c \rightarrow (s, c, p)$	Spend all available capital
Share-centric	$s \rightarrow (s, c, p)$	Portfolio rebalancing
Probability-centric	$p \rightarrow (s, c, p)$	Limit order execution

These map to a 2×2 design space: (1) execution order (arb first, or target first), and (2) redemption strategy (NO in others, or NO in all):

	NO in others $\eta(n - 2)$ cash back	NO in all $\eta(n - 1)$ cash back
Arb first, target second (target inside search)	Dollar-centric Input: cost c	(No clean parameterization)
Target first, arb second (target outside search)	Probability-centric Input: probability p	Share-centric Input: shares s

The fourth combination (arb first + NO in all) lacks clean semantics: the target purchase happens inside the search loop with remaining budget, but NO-in-all redemption doesn't provide cash for target purchases. We focus on the three useful parameterizations.

All three achieve $O(n \log N)$ complexity using the same core technique: direct cost evaluation eliminates nested searches.

2.2 Relationship to Redemption Strategies

Multi-outcome markets maintain liquidity pools (Y_i, N_i) for each outcome i , where Y_i represents YES share reserves and N_i represents NO share reserves. Traders buy YES shares to bet an outcome occurs, or NO shares to bet it doesn't. A complete set (one YES share in each outcome, or equivalently one NO share in each) always pays out exactly \$1.

Share redemption is central to auto-arbitrage. Holding η NO shares in each of n outcomes yields $\eta(n - 1)$ guaranteed payout (exactly $n - 1$ outcomes lose). This enables two approaches:

- **Redemption 1 (“NO in others”)**: Buy η NO in $(n - 1)$ non-target outcomes. Yields $\eta(n - 2)$ cash plus η YES in target. Used by share-centric and probability-centric.
- **Redemption 2 (“NO in all”)**: Buy η NO in all n outcomes. Yields $\eta(n - 1)$ cash. Used by dollar-centric.

3 Model

3.1 Market Structure

A market with n outcomes has pools $\{(Y_i, N_i)\}_{i=1}^n$ with probabilities:

$$p_i = f(Y_i, N_i), \quad \sum_{i=1}^n p_i = 1 \quad (1)$$

Each outcome has cost functions $C^Y(\delta)$ and $C^N(\eta)$ giving the cost to purchase δ YES shares or η NO shares from pool (Y, N) . Requirements:

- **Monotonic**: More shares cost more
- **Computable**: $C^Y(\cdot)$ and $C^N(\cdot)$ evaluable in $O(1)$ time, in closed form or via a bounded constant-iteration solve (see the CPMM example for when each applies)

Examples: CPMM, logarithmic scoring rules, hybrids with limit orders. Our algorithms work with any such curves.

3.2 CPMM Example

The constant-product market maker carries a weight $p \in (0, 1)$ (the pool weight, distinct from the outcome probabilities p_i), with invariant $k = Y^p N^{1-p}$ and probability

$$f(Y, N) = \frac{pN}{(1-p)Y + pN}, \quad (2)$$

which reduce to $k = \sqrt{YN}$ and $f = N/(Y + N)$ at $p = \frac{1}{2}$. The two directions of the cost map are *not* symmetric in p .¹

Cost-in is closed form for any p . Spending an amount A to buy YES sets $N' = N + A$ and restores the invariant via $Y' = (k/N'^{1-p})^{1/p}$, yielding $\delta = A + (Y - Y')$ shares. This is $O(1)$ for *any* p —two power evaluations, no equation solving.

Shares-in is closed form only at $p = \frac{1}{2}$. The inverse—the cost $C^Y(\delta)$ to buy a *given* δ —requires solving $(Y - \delta + C)^p (N + C)^{1-p} = k$ for C . At $p = \frac{1}{2}$ this collapses to a quadratic:

$$C^Y(\delta) = \frac{\delta - Y - N + \sqrt{(Y + N - \delta)^2 + 4N\delta}}{2}, \quad (3)$$

with the analogous $C^N(\eta)$ for NO. For general p it is transcendental: in log space the relation $p \log(Y - \delta + C) + (1-p) \log(N + C) = \log k$ is a sum of two logarithms with an irrational weight—not Lambert- W form, so no standard special function closes it. Clearing the fractional powers for rational $p = a/b$ leaves a degree- b polynomial in C ; already at $p = \frac{1}{5}$ that quintic has Galois group S_5 , so no radical solution exists.

Consequence for the algorithms. Prefer the cost-in parameterization—the dollar-centric strategy (Algorithm 1) is naturally cost-in and stays $O(1)$ for all p . Where a shares-in evaluation is unavoidable, replace the closed form with a bounded one-dimensional Newton iteration seeded from the $p = \frac{1}{2}$ formula; the map is monotone and convex, so it reaches machine precision in a handful of steps, preserving the $O(n \log N)$ bound up to a small constant.

4 Three Algorithms

4.1 Option 1: Dollar-Centric

Use case: Spend exactly a specified cost.

Notation: `BINARYSEARCH( $a, b, f$ )` finds $x \in [a, b]$ where $f(x) = 0$ via bisection.

¹The claims in this subsection are derived and machine-checked symbolically in the accompanying proof scripts `tasks/cpmm_multi_2/proofs/general_p_cost.py` (GP1–GP4).

Algorithm 1 Dollar-Centric Purchase

Input: Answer k , cost c , pools $\{(Y_i, N_i)\}_{i=1}^n$

Output: (s, c, p) triple

```
1:  $\eta \leftarrow \text{BINARYSEARCH}(0, \eta_{\max}, \text{CHECKBUDGET})$ 
2: function CHECKBUDGET( $\eta$ )
3:    $c_{\text{arb}} \leftarrow \sum_{i \neq k} C^N(\eta \mid Y_i, N_i)$  ▷ NO in others only
4:   redemption  $\leftarrow \eta \cdot (n - 2)$  ▷ NO in others
5:    $c_{\text{net}} \leftarrow c_{\text{arb}} - \text{redemption}$ 
6:    $c_{\text{primary}} \leftarrow c - c_{\text{net}}$ 
7:   Buy  $\delta$  YES in  $k$  using  $c_{\text{primary}}$ , update pools
8:   return  $\sum p'_i - 1$  ▷ Check balance
9: end function
10: return  $(s = \delta + \eta, c, p')$ 
```

Complexity: $O(n \log N)$

4.2 Option 2: Share-Centric

Use case: Purchase a specific number of shares.

Key insight: Buy exact shares in target first (outside search). Then search for η in all n answers to restore balance.

Algorithm 2 Share-Centric Purchase

Input: Answer k , target shares s , pools $\{(Y_i, N_i)\}_{i=1}^n$

Output: (s, c, p) triple

```
1:  $c_{\text{primary}} \leftarrow C^Y(s \mid Y_k, N_k)$ 
2: Buy  $s$  YES in  $k$ , update pool  $(Y'_k, N'_k)$ 
3:
4:  $\eta \leftarrow \text{BINARYSEARCH}(0, \eta_{\max}, \text{CHECKBALANCE})$ 
5: function CHECKBALANCE( $\eta$ )
6:    $c_{\text{arb}} \leftarrow \sum_{i=1}^n C^N(\eta \mid Y'_i, N'_i)$  ▷ NO in all  $n$  answers
7:   Apply NO purchases, compute  $\{p'_i\}$ 
8:   return  $\sum p'_i - 1$ 
9: end function
10:  $c \leftarrow c_{\text{primary}} + c_{\text{arb}} - \eta(n - 1)$  ▷ Redemption
11: return  $(s, c, p')$ 
```

Complexity: $O(n \log N)$

4.3 Option 3: Probability-Centric

Use case: Move target to a specific probability (limit orders).

Key insight: Buy in target to reach p_{target} first (outside search). Then search for η NO in *others only*. Since redemption is pure accounting and doesn't modify target pool, probability stays fixed at p_{target} .

Algorithm 3 Probability-Centric Purchase

Input: Answer k , probability p_{target} , pools $\{(Y_i, N_i)\}_{i=1}^n$

Output: (s, c, p) triple with $p = p_{\text{target}}$

```
1:  $\delta \leftarrow \text{SHARES\_FOR\_PROB}(Y_k, N_k, p_{\text{target}})$  ▷ Invert  $p = f(Y, N)$ 
2:  $c_{\text{primary}} \leftarrow C^Y(\delta \mid Y_k, N_k)$ 
3: Buy  $\delta$  YES in  $k$ , update  $(Y'_k, N'_k)$ 
4: Verify  $f(Y'_k, N'_k) = p_{\text{target}}$ 
5:
6:  $\eta \leftarrow \text{BINARY\_SEARCH}(0, \eta_{\text{max}}, \text{CHECK\_BALANCE})$ 
7: function CHECKBALANCE( $\eta$ )
8:    $c_{\text{arb}} \leftarrow \sum_{i \neq k} C^N(\eta \mid Y_i, N_i)$  ▷ NO in others only
9:   Apply NO purchases, compute  $\{p'_i\}_{i \neq k}$ 
10:  return  $p_{\text{target}} + \sum_{i \neq k} p'_i - 1$  ▷ Target fixed
11: end function
12:  $c \leftarrow c_{\text{primary}} + c_{\text{arb}} - \eta(n - 2)$  ▷ Redemption
13: return  $(s = \delta + \eta, c, p = p_{\text{target}})$ 
```

Proposition 4.1 (Target Probability Invariance). *The probability of answer k remains p_{target} throughout auto-arbitrage and after redemption.*

Proof. Auto-arbitrage modifies only pools $\{(Y_i, N_i)\}_{i \neq k}$. Since $p_k = f(Y'_k, N'_k)$ depends only on (Y'_k, N'_k) , which is unchanged, $p_k = p_{\text{target}}$ holds. Redemption is share identity application (pure accounting), not pool modification. $\square$

Complexity: $O(n \log N)$

Application: Essential for limit orders specified by target probability, enabling efficient partial fills with guaranteed final price.

5 Comparison to Nested Search

Existing implementations invert the cost function via nested search:

Algorithm 4 Nested Search Approach (Existing)

Input: Answer k , cost c , pools

```
1:  $\eta \leftarrow \text{BINARY\_SEARCH}(0, \eta_{\text{max}}, \text{OUTER\_CHECK})$  ▷  $O(\log N)$ 
2: function OUTERCHECK( $\eta$ )
3:    $c_{\text{arb}} \leftarrow \sum_{i \neq k} \text{SHARES\_TO\_COST}(\eta, Y_i, N_i)$  ▷ Each term:  $O(\log N)$  search!
4:    $c_{\text{primary}} \leftarrow c - (c_{\text{arb}} - \eta(n - 2))$ 
5:   Buy YES in  $k$  with  $c_{\text{primary}}$ , compute  $\{p'_i\}$ 
6:   return  $\sum p'_i - 1$ 
7: end function
```

Complexity: $O(n \log^2 N)$ — Outer $O(\log N)$ times $(n \times O(\log N))$ inversions.

Speedup: Direct formulas eliminate inner searches:

$$\frac{O(n \log^2 N)}{O(n \log N)} = O(\log N) \approx 20 \times \text{ for } \epsilon = 10^{-6} \quad (4)$$

Empirically: individual optimizer test cases taking minutes (the equivalent-set/exhaustive-fallback snapshot cases run ~ 250 s each) collapse to well under a second on the closed-form path.

6 Implementation Notes

6.1 Generality

Our algorithms require only:

1. Cost functions $C^Y(\delta)$ and $C^N(\eta)$ computable in $O(1)$
2. Probability function $p = f(Y, N)$ computable in $O(1)$
3. For probability-centric: Invertible $\delta = f^{-1}(p)$ computable in $O(1)$

This works with CPMM (any weight p), logarithmic scoring, or hybrid mechanisms incorporating limit orders. The key is avoiding nested searches for cost/share inversion. For CPMM the cost-in direction is closed form for every p , while shares-in is closed form only at $p = \frac{1}{2}$ (Section 3.2); away from $\frac{1}{2}$, requirement 1 is met by a bounded Newton inversion rather than a radical formula, which suffices because it still removes the inner *search* layer.

6.2 Reversibility with limit orders

The binary searches above bracket their target from both sides, so during convergence an outcome's probability may move up past a price and then back down. Correctness therefore requires the cost map to be *reversible*: $C(\delta) + C(-\delta) = 0$. Pure CPMM satisfies this—the cost is a state function of the pool, so a round trip is exactly zero. Limit order books break it under the obvious implementation, in which crossing a maker order consumes it in place and never restores it (a ratchet). An overshoot probe then consumes makers that the *settled* bet never crosses, and the computed result depends on the search trajectory rather than on the net bet.

It is worth distinguishing two things. A *real* round-trip trade—buy up through some orders, later sell back down through others—crosses different makers each way and is legitimately irreversible: two real trades, real spread paid. That is correct. What must be reversible is only the auto-arbitrage *search's internal trajectory*, since the search computes a single net bet and its result must depend only on the net movement per outcome.

The fix is to leave a **temporary reverse limit** for the duration of the atomic operation: when a probe crosses (or partially crosses) a maker at price ρ , insert an opposite-side order of the consumed size at the same price ρ . By price priority a returning probe re-crosses that reverse order first and is refunded exactly what it paid, restoring the maker. This keeps the real limit book live in the search—so every probe is priced with limits included, unlike a pure-CPMM proxy—while making the consumed amount of every maker a pure function of the current price. Taker cash is then a potential $\Phi(p)$ with $C(p_a \rightarrow p_b) = \Phi(p_b) - \Phi(p_a)$, so reversibility ($C(\delta) + C(-\delta) = 0$) and path-independence of the search are identities rather than approximations. At commit the reverse limits are discarded, leaving exactly the makers crossed by the net movement; on a monotone (non-overshooting) path the procedure is identical to a single direct bet, so it is corrective only on the pathological trajectories it is designed to neutralize.

6.3 Monotonicity of the multi-buy equilibrium

The reverse-limit construction above neutralizes a *search* that overshoots. A separate and sharper question is whether the *equilibrium itself* moves monotonically with the bet size—because if it does, a correct implementation need never overshoot at all, and the limit-fill semantics are unambiguous (each maker is crossed once, in one direction). This matters for the one auto-arbitrage path that is genuinely non-monotone in its naive form: a *basket buy* of YES across several outcomes, implemented as an iterate–buy–arbitrage–reinvest loop. Such a loop drives a traded outcome’s probability up past its settled value and then back down within a single atomic operation, so a resting order in that transient band is consumed on the way up and—under in-place settlement—kept on the way down.

Fix a sum-to-one CPMM market at weight $p = \frac{1}{2}$. A basket buy purchases an equal number g of YES shares in each outcome of a set $B \subsetneq \{1, \dots, n\}$; the auto-arbitrage buys an equal number η of NO shares in all n outcomes to restore $\sum_i p_i = 1$ and redeems the η complete NO-sets for $\eta(n-1)$. Write t for the net mana spent.

Theorem 6.1 (Monotone multi-buy equilibrium). *For every $t \geq 0$ the equilibrium (g, η) exists and is unique, and each outcome probability is monotone in t : strictly increasing for $i \in B$ and strictly decreasing for $i \notin B$. Consequently each outcome’s price reaches its settled value by a single monotone move—no overshoot.*

Proof sketch. The argument rests on one identity. At $p = \frac{1}{2}$ the marginal YES-buy and NO-buy rates on a pool coincide in magnitude:

$$a_i := \frac{\partial p_i}{\partial(\text{YES share})} = \frac{2Y_i N_i}{(Y_i + N_i)^3} = -\frac{\partial p_i}{\partial(\text{NO share})}. \quad (5)$$

Existence and uniqueness of the arbitrage η at fixed g are immediate: buying NO strictly lowers each probability, so $\sum_i p_i(\eta)$ is strictly decreasing, giving a unique root of $\sum_i p_i = 1$ (this part holds for any p). Differentiating that balance $F(g, \eta) = \sum_i p_i - 1 = 0$ implicitly, and using $a_i = b_i$, every comparative static signs in closed form:

$$\frac{d\eta}{dg} = \frac{\sum_{l \in B} a_l}{\sum_k a_k} \in (0, 1), \quad \frac{dp_i}{dg} = \begin{cases} a_i \frac{\sum_{j \notin B} a_j}{\sum_k a_k} > 0 & i \in B, \\ -a_i \frac{\sum_{l \in B} a_l}{\sum_k a_k} < 0 & i \notin B, \end{cases} \quad (6)$$

with $\sum_{j \notin B} a_j > 0$ because B is a proper subset. Finally the net spend satisfies

$$\frac{dt}{dg} = \sum_{i \in B} p_i > 0, \quad (7)$$

the $\eta(n-1)$ redemption exactly cancelling the arbitrage’s marginal cost (this is precisely the “yes share price sum” that existing implementations divide by). So t is strictly increasing in g , the map is invertible, and each p_i inherits strict monotonicity in t . $\square$

The payoff is for limit orders. By Theorem 6.1 the settled price path of every outcome is monotone, so each resting maker is crossed exactly once, in one direction. The ordinary single-move limit fill—which already pins correctly, a price resting *at* a large order—therefore gives the right answer; the transient overshoot of the iterate-reinvest loop is a provable algorithm *artifact*,

not a property of the equilibrium. A direct monotone solve and the reverse-limit settlement of the previous subsection both realize this same unique equilibrium, so the choice between them is one of implementation, not correctness.²

6.4 Multi-target redemption and conservation

The redemption identities of §2.2 (“Relationship to Redemption Strategies”) were stated for a *single* target: “NO in others” buys η NO in the $n - 1$ non-target outcomes and credits $\eta(n - 2)$ cash plus η YES in the target. A basket buy bets YES on a set B with $|B| = m \geq 2$ outcomes at once, and the credit must be distributed across the m targets. This generalization is not innocuous: the wrong split silently destroys mana at resolution.

Take the “NO in others” decomposition for a basket: buy an equal g YES in each $i \in B$ and an equal η NO in each of the $n - m$ non-basket outcomes. The held NO bundle $\{\eta$ NO in each non-basket outcome $\}$ pays $\eta(n - m)$ when a basket outcome wins and $\eta(n - m - 1)$ when a non-basket outcome wins. Its guaranteed floor $\eta(n - m - 1)$ is redeemed for cash (the $m = 1$ case recovers $\eta(n - 2)$). The *residual*—value beyond the floor—pays η exactly when some basket outcome wins, and 0 otherwise. Expressed as YES shares s_i on the basket outcomes, a claim that pays η for *whichever* $i \in B$ wins requires $s_i = \eta$ for every $i \in B$ (only the winning outcome’s YES shares pay). The naive even split $s_i = \eta/m$ pays a winning basket outcome only η/m .

Theorem 6.2 (Basket redemption credit). *In a sum-to-one market, resolution conserves value—total payout independent of which outcome wins—if and only if $D_i := T_i^{\text{YES}} - T_i^{\text{NO}}$ is constant across outcomes, where $T_i^{\text{YES}} = Y_i + (\text{trader YES shares in } i)$ and $T_i^{\text{NO}} = N_i + (\text{trader NO shares in } i)$. A basket buy preserves this invariant if and only if each basket outcome is credited η YES shares (not η/m). The condition is independent of the pool weights p_i .*

Proof sketch. Resolution to winner W pays $T_W^{\text{YES}} + \sum_{j \neq W} T_j^{\text{NO}}$ (the winner’s YES claims plus every other outcome’s NO claims, pools and trader shares alike). This is constant in W iff $T_i^{\text{YES}} - T_i^{\text{NO}}$ is constant in i . A CPMM buy moves $Y_i - N_i$ by exactly $-s$ for a YES purchase of s shares and $+s$ for a NO purchase (the share mechanic $Y' = Y + b - s$, $N' = N + b$ gives $\Delta(Y - N) = -s$; symmetrically for NO; no dependence on p or amount b). For a basket outcome credited c YES shares, $\Delta D_i = (-g) + (g + c) = c$; for a non-basket outcome with its NO shares redeemed away, $\Delta D_j = +\eta$. The pre-bet D is uniform (the sum-to-one funding identity), so it stays uniform iff $c = \eta$. $\square$

For $m = 1$, $\eta/m = \eta$, so the single-target algorithms of §4 were always correct; the defect is specific to genuine multi-target baskets and to resolution *onto a basket outcome* (resolving to a non-basket outcome conserves regardless, which is why it can hide).³

²Stated and machine-checked—symbolically for the marginal identity and comparative statics, numerically for the equilibrium solve and the limit-fill behavior—in `tasks/cpmm_multi_2/proofs/monotone_equilibrium.py` (GP12), against a faithful reconstruction of the vendor basket-buy loop. The marginal symmetry $a_i = b_i$ is special to $p = \frac{1}{2}$; the non-basket and basket-sum monotonicity and the single-crossing limit argument carry to any p , and the individual-basket case for general p is numerically confirmed with formalization pending.

³This was a live bug in a `cpmm-multi-2` implementation that split the credit η/m ; it destroyed $\sim 10\%$ of the stake at resolution to a basket outcome and was caught only by an end-to-end conservation test on a running instance (a v1 market on the identical bet conserved). Stated and machine-checked—symbolically for the pool mechanic, the residual decomposition, and the conservation argument; numerically end-to-end for the leak and its fix—in `tasks/cpmm_multi_2/proofs/multi_target_redemption.py` (GP16).

6.5 Convergence

Binary search typically converges in 18–24 iterations for tolerance $\epsilon = 10^{-6}$. Each iteration evaluates direct formulas on $O(n)$ answers, totaling $\sim 400n$ arithmetic operations for typical $n = 10$.

7 Pool Initialization under General Weights

Freeing the per-answer weight does more than allow non-uniform starting probabilities; it opens a design choice at *market creation*. A sum-to-one market is funded by an ante budget $\mathcal{A}$, and the creator must place n pools $\{(Y_i, N_i, w_i)\}$ that (i) read off the target probabilities p_i , (ii) never let the house lose money, and (iii) are as *liquid* as possible. The legacy uniform construction pins $w_i = \frac{1}{2}$ and is forced to an asymmetric pool; with a free w_i the question becomes a genuine optimization. This section settles the right liquidity metric, the funding constraint, and the optimal allocation.⁴

7.1 The right liquidity metric

For one pool (Y, N) with weight w and probability $\rho = wN/((1-w)Y + wN)$, the marginal price impact of a trade—differential probability movement per share—has the closed form

$$a \equiv \frac{d\rho}{d\delta} = \frac{w(1-w)YN}{W^3} = \frac{\rho(1-\rho)}{W}, \quad W \equiv (1-w)Y + wN, \quad (8)$$

reducing to the symmetric $a = 2YN/(Y+N)^3$ at $w = \frac{1}{2}$. Crucially a is measured *per share*, not per unit of currency: in shares the impact is invariant under the swap $(Y, N, w) \mapsto (N, Y, 1-w)$ (buying YES and buying NO are equally easy), whereas the per-currency impact is not, since the marginal prices of a YES and a NO share are ρ and $1-\rho$. Eliminating w at fixed probability gives $a = \rho(1-\rho)(\rho Y + (1-\rho)N)/(YN)$. The immediate consequence: **at a fixed probability, maximizing liquidity is exactly maximizing $W = (1-w)Y + wN$** . This corrects the stored “liquidity” $\sqrt{YN}$, which is the wrong yardstick once $w \neq \frac{1}{2}$.

Under the basket auto-arbitrage, a single-answer YES buy is partly absorbed by the η complete-set-of-NO redemption that restores $\sum_i p_i = 1$. The effective slope is again closed form,

$$a_i^{\text{eff}} = a_i \frac{\sum_j a_j - a_i}{\sum_j a_j}, \quad (9)$$

which is monotone in a_i ; optimizing the creation pools against a^{eff} rather than a moves the optimum by only one to three percentage points, so the pre-arbitrage metric (8) is a faithful proxy.

7.2 Funding and the optimal allocation

Exactly one outcome resolves YES, so if outcome k wins the house pays $Y_k + \sum_{j \neq k} N_j$. No-house-loss requires this to not exceed the ante for every k . At the liquidity optimum every such scenario binds, which forces

$$Y_i - N_i = D \quad (\text{constant in } i), \quad D + \sum_i N_i = \mathcal{A}. \quad (10)$$

Substituting (10) turns the design into an *unconstrained* minimization of $\sum_i a_i$ over the NO-reserves $\{N_i\}$ (with $D = \mathcal{A} - \sum_i N_i$), which is well behaved and solved by a small Newton step. The

⁴Derived and machine-checked in `tasks/cpmm_multi.2/proofs/creation_liquidity_proofs.py` (GP13–GP15, symbolic) with numerical companions `creation_liquidity\{_, optimum, solver\}.py`.

balanced pool $Y_i = N_i$ is *not* optimal except at uniform: the optimum is asymmetric—deeper on each answer’s cheaper side, with w_i pushed off p_i —and is 11% more liquid at $n = 3$, rising past 50% at $n = 6$.

A clean closed form recovers almost all of this gain. Minimizing $\sum_i \rho_i(1 - \rho_i)/W_i$ subject to a budget on $\sum_i W_i$ gives, by Cauchy–Schwarz, $W_i \propto \sqrt{\rho_i(1 - \rho_i)}$. Since $\rho_i(1 - \rho_i)$ is the variance of the outcome’s Bernoulli indicator, this is the familiar rule

$$\boxed{\text{allocate pool depth } W_i \propto \sqrt{\rho_i(1 - \rho_i)}} \tag{11}$$

—depth proportional to the outcome’s standard deviation. Across configurations it captures 95.5–99.7% of the balanced-to-optimal liquidity gain, so it is usable directly or as a seed for the exact solve.

7.3 Independent (non-sum-to-one) markets

When the answers do not sum to one, each is a standalone CPMM funded by its own share of the ante, and the binding budget is the per-answer maximum loss $\max(Y_i, N_i)$. At fixed maximum loss the liquidity-optimal shape is the *balanced* pool $Y_i = N_i$ with $w_i = p_i$ —precisely the binary-market construction. Independent multi-answer markets should therefore initialize each answer exactly as a binary CPMM does; the asymmetric optimum above is specific to the shared-funding structure of sum-to-one markets.

8 Conclusion

We presented three $O(n \log N)$ parameterization strategies for multi-outcome market auto-arbitrage, achieving $\sim 20\times$ speedup over nested $O(n \log^2 N)$ approaches. The key insight: direct cost formulas eliminate inner binary searches. Our approach is general, working with any smooth monotonic cost curves.

The probability-centric strategy is particularly valuable for limit orders, guaranteeing exact final probability for efficient partial fills. All three strategies are production-ready and validated through extensive testing.